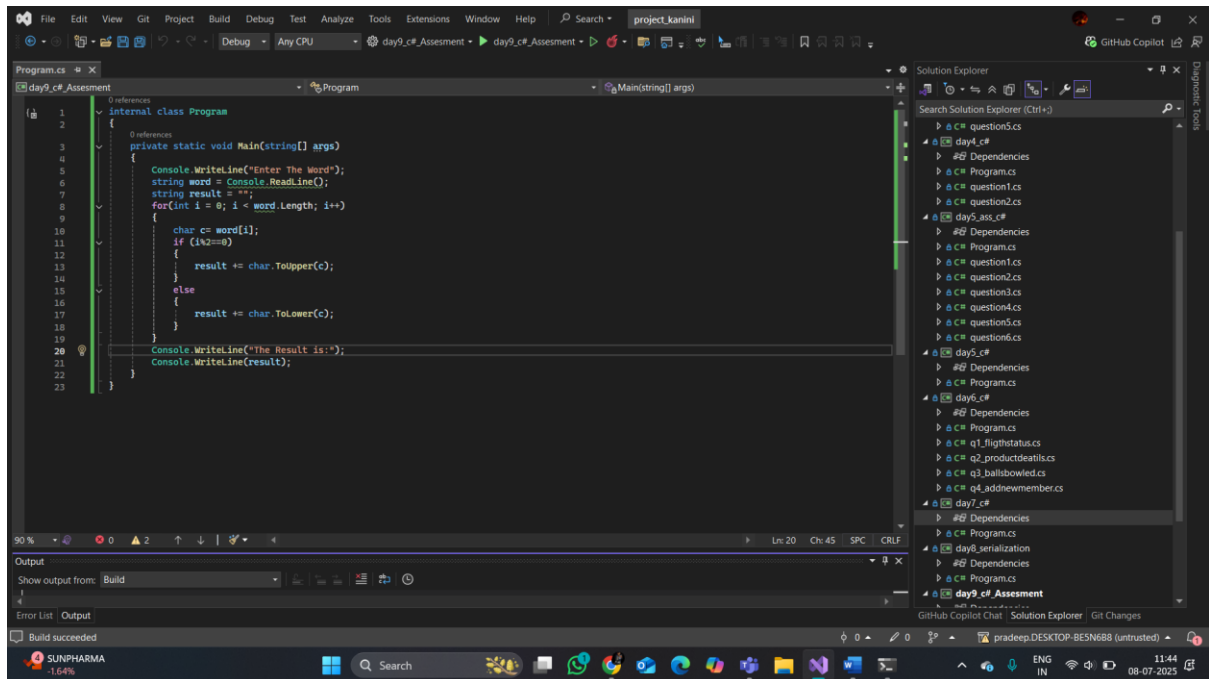


Pradeepkumar M

C# Assessment :

Question 1 : convert the characters of the word into upper and lower-case letters vice versa.

MARK : 30

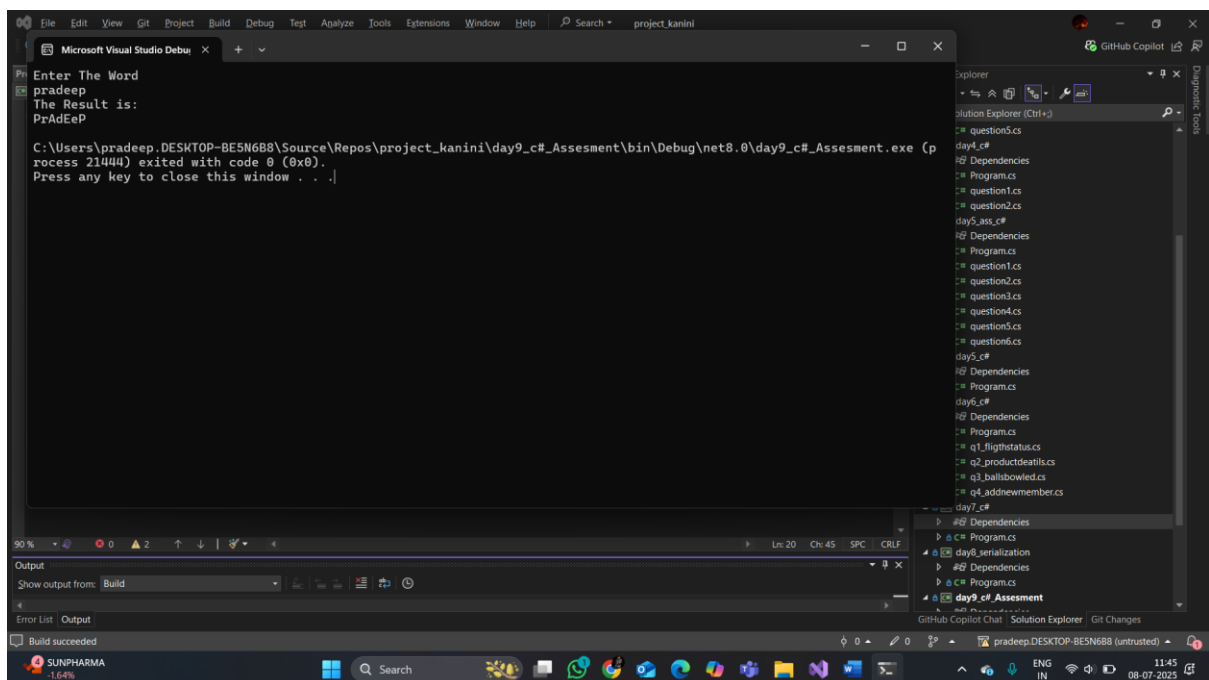


The screenshot shows the Visual Studio IDE with a C# project named 'project_kanini'. The file 'day9_c#_Assesment' is open, displaying the following code:

```
1 internal class Program
2 {
3     private static void Main(string[] args)
4     {
5         Console.WriteLine("Enter The Word");
6         string word = Console.ReadLine();
7         string result = "";
8         for(int i = 0; i < word.Length; i++)
9         {
10             char c = word[i];
11             if (i % 2 == 0)
12             {
13                 result += char.ToUpper(c);
14             }
15             else
16             {
17                 result += char.ToLower(c);
18             }
19         }
20         Console.WriteLine("The Result is:");
21         Console.WriteLine(result);
22     }
23 }
```

The Solution Explorer on the right shows the project structure, including 'day9_c#_Assesment' and its dependencies. The status bar at the bottom indicates 'Build succeeded'.

Output:



The screenshot shows the Visual Studio IDE with the 'Microsoft Visual Studio Debug' window open. The output of the program is displayed as follows:

```
Enter The Word
pradeep
The Result is:
PrAdEeP

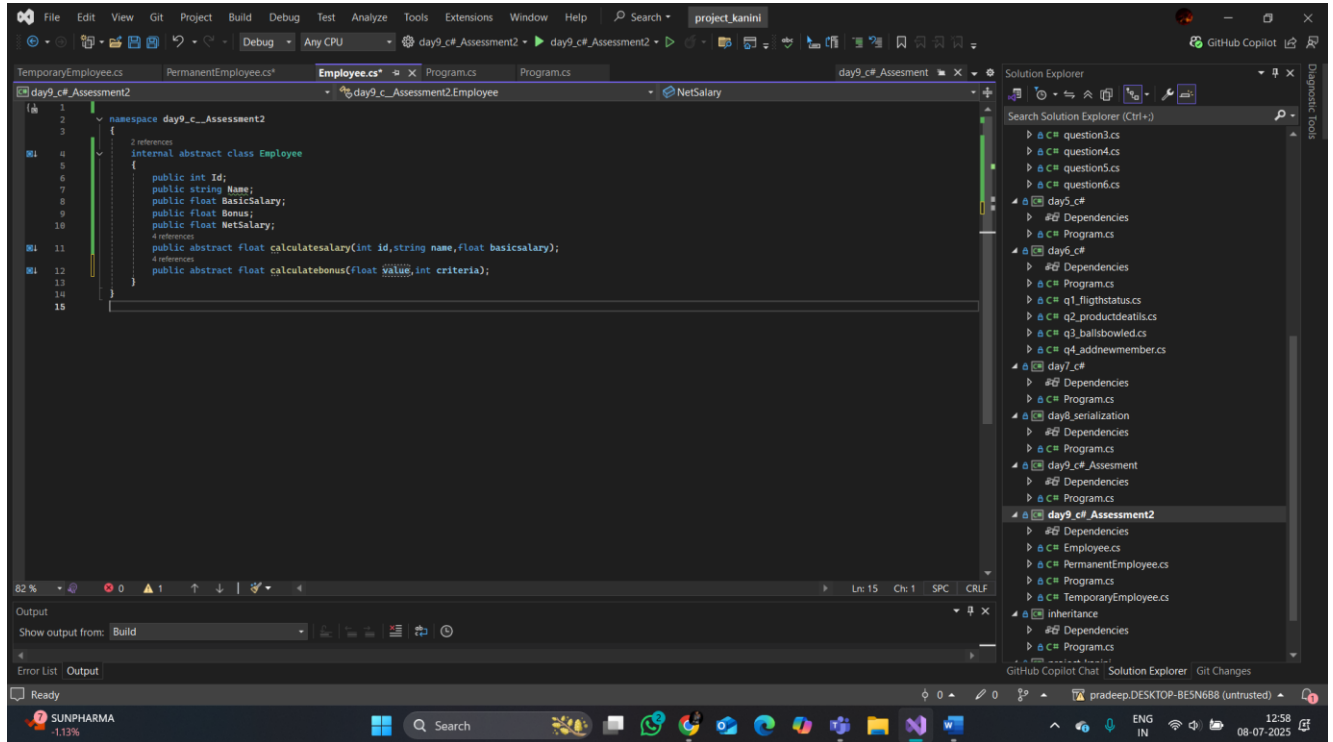
C:\Users\pradeep.DESKTOP-BE5N6B8\Source\Repos\project_kanini\day9_c#_Assesment\bin\Debug\net8.0\day9_c#_Assesment.exe (p
rocess 21444) exited with code 0 (0x0).
Press any key to close this window . . .
```

The Solution Explorer on the right shows the project structure, including 'day9_c#_Assesment' and its dependencies. The status bar at the bottom indicates 'Build succeeded'.

Question 2: Employee Payroll Management

MARK : 40

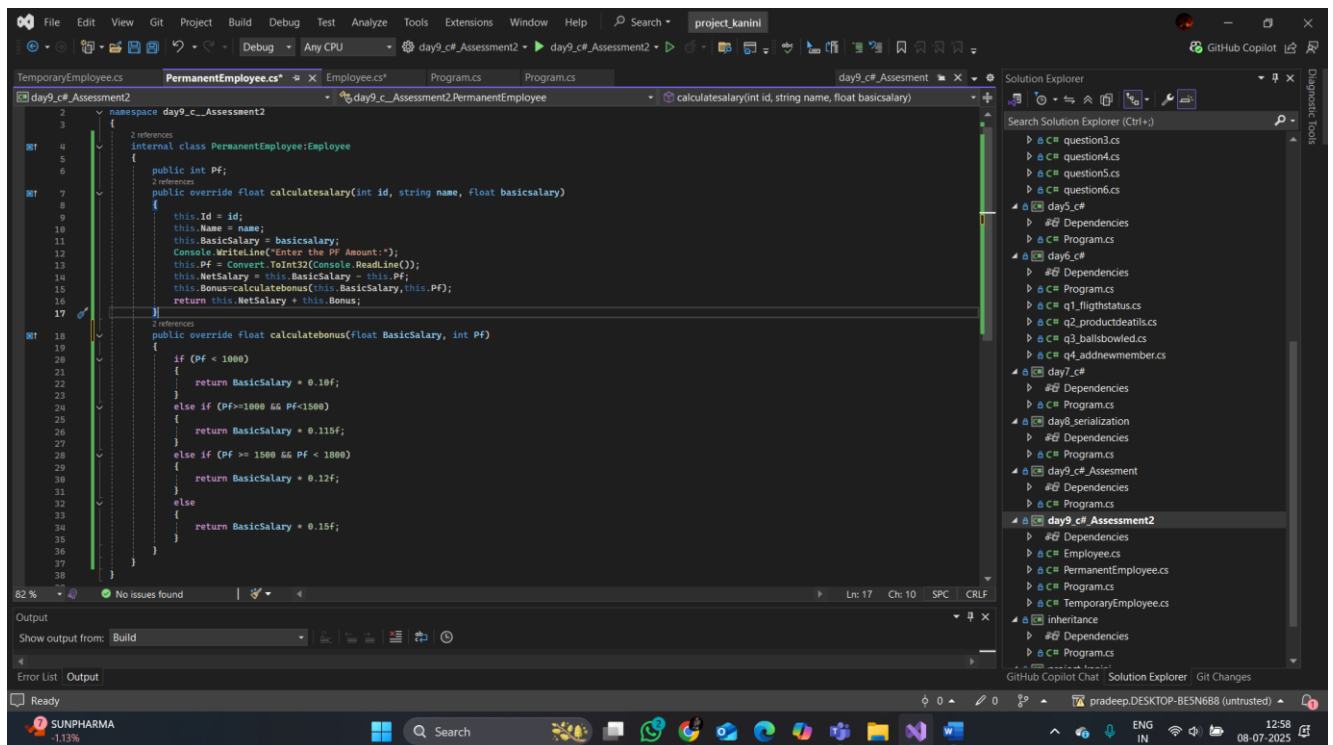
Abstract Employee class



The screenshot shows the Visual Studio IDE with the 'Employee.cs' file open. The code defines an abstract class 'Employee' within the 'day9_c_Assessment2' namespace. The class has several properties: 'Id' (int), 'Name' (string), 'BasicSalary' (float), 'Bonus' (float), and 'NetSalary' (float). It also has two abstract methods: 'calculatesalary' and 'calculatebonus'. The 'Solution Explorer' on the right shows the project structure, including 'day9_c_Assessment2' and its sub-projects 'Employee.cs', 'PermanentEmployee.cs', and 'TemporaryEmployee.cs'.

```
1 namespace day9_c_Assessment2
2 {
3     2 references
4     internal abstract class Employee
5     {
6         public int Id;
7         public string Name;
8         public float BasicSalary;
9         public float Bonus;
10        public float NetSalary;
11        4 references
12        public abstract float calculatesalary(int id, string name, float basicsalary);
13        4 references
14        public abstract float calculatebonus(float value, int criteria);
15    }
16 }
```

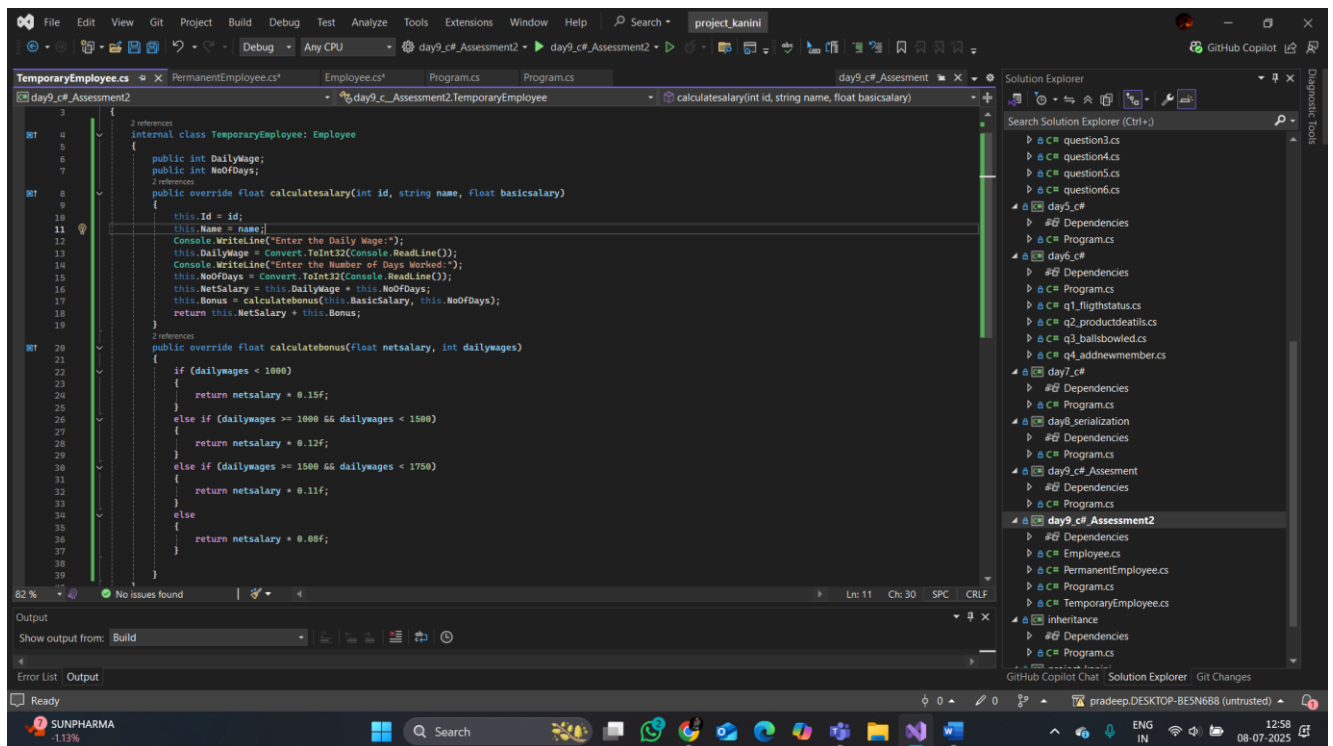
Permanent Employee Class:



The screenshot shows the Visual Studio IDE with the 'PermanentEmployee.cs' file open. The code defines a class 'PermanentEmployee' within the 'day9_c_Assessment2' namespace, which inherits from the 'Employee' class. It has a property 'PF' (int) and overrides the 'calculatesalary' and 'calculatebonus' methods. The 'calculatesalary' method calculates the net salary based on the basic salary and PF. The 'calculatebonus' method calculates the bonus based on the basic salary and PF. The 'Solution Explorer' on the right shows the project structure, including 'day9_c_Assessment2' and its sub-projects 'Employee.cs', 'PermanentEmployee.cs', and 'TemporaryEmployee.cs'.

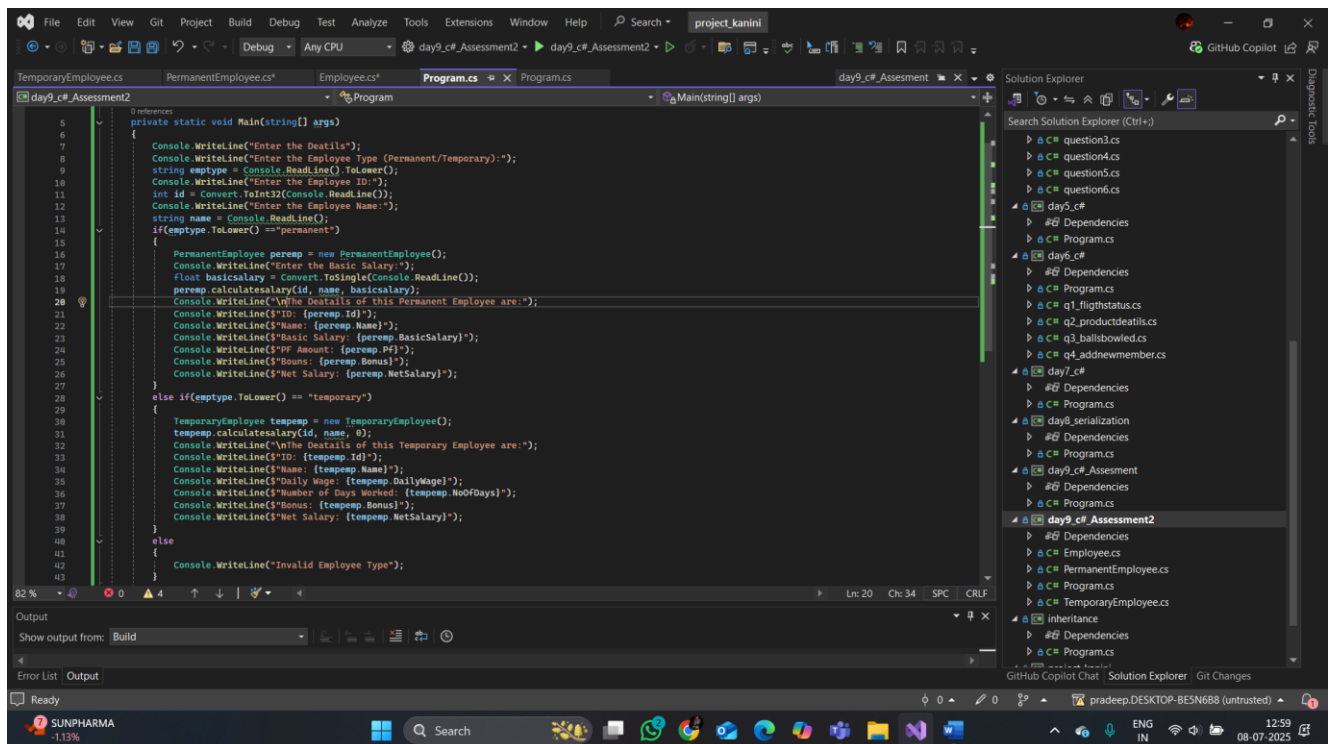
```
1 namespace day9_c_Assessment2
2 {
3     2 references
4     internal class PermanentEmployee:Employee
5     {
6         public int PF;
7         2 references
8         public override float calculatesalary(int id, string name, float basicsalary)
9         {
10            this.Id = id;
11            this.Name = name;
12            this.BasicSalary = basicsalary;
13            Console.WriteLine("Enter the PF Amount:");
14            this.PF = Convert.ToInt32(Console.ReadLine());
15            this.NetSalary = this.BasicSalary + this.PF;
16            this.Bonus = calculatebonus(this.BasicSalary, this.PF);
17            return this.NetSalary + this.Bonus;
18        }
19        2 references
20        public override float calculatebonus(float BasicSalary, int PF)
21        {
22            if (PF < 1000)
23            {
24                return BasicSalary * 0.10f;
25            }
26            else if (PF >= 1000 && PF < 1500)
27            {
28                return BasicSalary * 0.115f;
29            }
30            else if (PF >= 1500 && PF < 1800)
31            {
32                return BasicSalary * 0.12f;
33            }
34            else
35            {
36                return BasicSalary * 0.15f;
37            }
38        }
39    }
40 }
```

Temporary Employee Class:



```
3 // 2 references
4 internal class TemporaryEmployee : Employee
5 {
6     public int DailyWage;
7     public int NoOfDays;
8 // 2 references
9     public override float calculatesalary(int id, string name, float basicsalary)
10     {
11         this.Id = id;
12         this.Name = name;
13         Console.WriteLine("Enter the Daily Wage:");
14         this.DailyWage = Convert.ToInt32(Console.ReadLine());
15         Console.WriteLine("Enter the Number of Days Worked:");
16         this.NoOfDays = Convert.ToInt32(Console.ReadLine());
17         this.NetSalary = this.DailyWage * this.NoOfDays;
18         this.Bonus = calculatebonus(this.BasicSalary, this.NoOfDays);
19         return this.NetSalary + this.Bonus;
20     }
21 // 2 references
22     public override float calculatebonus(float netsalary, int dailywages)
23     {
24         if (dailywages < 1000)
25         {
26             return netsalary * 0.15f;
27         }
28         else if (dailywages >= 1000 && dailywages < 1500)
29         {
30             return netsalary * 0.12f;
31         }
32         else if (dailywages >= 1500 && dailywages < 1750)
33         {
34             return netsalary * 0.11f;
35         }
36         else
37         {
38             return netsalary * 0.08f;
39         }
40     }
41 }
```

Main Class :



```
8 // 0 references
9 private static void Main(string[] args)
10 {
11     Console.WriteLine("Enter the Details");
12     Console.WriteLine("Enter the Employee Type (Permanent/Temporary):");
13     string emtype = Console.ReadLine().ToLower();
14     Console.WriteLine("Enter the Employee ID:");
15     int id = Convert.ToInt32(Console.ReadLine());
16     Console.WriteLine("Enter the Employee Name:");
17     string name = Console.ReadLine();
18     if (emtype.ToLower() == "permanent")
19     {
20         PermanentEmployee peremp = new PermanentEmployee();
21         Console.WriteLine("Enter the Basic Salary:");
22         float basicsalary = Convert.ToSingle(Console.ReadLine());
23         peremp.calculatesalary(id, name, basicsalary);
24         Console.WriteLine("\nthe Details of this Permanent Employee are:");
25         Console.WriteLine($"Id: {peremp.Id}");
26         Console.WriteLine($"Name: {peremp.Name}");
27         Console.WriteLine($"Basic Salary: {peremp.BasicSalary}");
28         Console.WriteLine($"PF Amount: {peremp.Pf}");
29         Console.WriteLine($"Bonus: {peremp.Bonus}");
30         Console.WriteLine($"Net Salary: {peremp.NetSalary}");
31     }
32     else if (emtype.ToLower() == "temporary")
33     {
34         TemporaryEmployee tempemp = new TemporaryEmployee();
35         tempemp.calculatesalary(id, name, 0);
36         Console.WriteLine("\nthe Details of this Temporary Employee are:");
37         Console.WriteLine($"Id: {tempemp.Id}");
38         Console.WriteLine($"Name: {tempemp.Name}");
39         Console.WriteLine($"Daily Wage: {tempemp.DailyWage}");
40         Console.WriteLine($"Number of Days Worked: {tempemp.NoOfDays}");
41         Console.WriteLine($"Bonus: {tempemp.Bonus}");
42         Console.WriteLine($"Net Salary: {tempemp.NetSalary}");
43     }
44     else
45     {
46         Console.WriteLine("Invalid Employee Type");
47     }
48 }
```

Output (permanent Employee):

The screenshot shows the Visual Studio IDE with the 'day9_c#_Assesment2' project open. The 'Program.cs' file is active, and the output window displays the following text:

```
Enter the Deatils
Enter the Employee Type (Permanent/Temporary):
permanent
Enter the Employee ID:
123
Enter the Employee Name:
pradeep
Enter the Basic Salary:
45000
Enter the PF Amount:
4500

The Deatails of this Permanent Employee are:
ID: 123
Name: pradeep
Basic Salary: 45000
PF Amount: 4500
Bouns: 6750.0005
Net Salary: 40500

C:\Users\pradeep.DESKTOP-BE5N6B8\Source\Repos\project_kanini\day9_c#_Assesment2\bin\Debug\net8.0\day9_c#_Assesment2.exe (process 14084) exited with code 0 (0x0).
Press any key to close this window . . .
```

The Solution Explorer on the right shows the project structure, including 'TemporaryEmployee.cs', 'PermanentEmployee.cs', 'Employee.cs', and 'Program.cs'. The status bar at the bottom indicates 'Build succeeded'.

Output (Temporary Employee):

The screenshot shows the Visual Studio IDE with the 'day9_c#_Assesment2' project open. The 'Program.cs' file is active, and the output window displays the following text:

```
Enter the Deatils
Enter the Employee Type (Permanent/Temporary):
temporary
Enter the Employee ID:
23
Enter the Employee Name:
pradeep
Enter the Daily Wage:
500
Enter the Number of Days Worked:
30

The Deatails of this Temporary Employee are:
ID: 23
Name: pradeep
Daily Wage: 500
Number of Days Worked: 30
Bonus: 0
Net Salary: 15000

C:\Users\pradeep.DESKTOP-BE5N6B8\Source\Repos\project_kanini\day9_c#_Assesment2\bin\Debug\net8.0\day9_c#_Assesment2.exe (process 28180) exited with code 0 (0x0).
Press any key to close this window . . .
```

The Solution Explorer on the right shows the project structure, including 'TemporaryEmployee.cs', 'PermanentEmployee.cs', 'Employee.cs', and 'Program.cs'. The status bar at the bottom indicates 'Build succeeded'.